IND320 Project Part-2

October 20, 2026

1 IND320 Project Work [Part-2]

1.1 Data Sources

Student name: Emre Can Emer

GitHub Repository link: https://github.com/Emriey/IND320_project

StreamLit Share link: https://emriey-ind320-project.streamlit.app/

Screencast and .pdf files can be found in the GitHub repository.

1.2 AI Usage

Google Gemini was used for this assignment. I normally prefer to ask small questions if I feel like I'm stuck and look for guidance rather than direct solutions.

I do not have copilot enabled in VsCode.

A few examples of the questions I asked AI:

- How to fix the notorious Py4JJavaError?
- Trying hopelessly fix the many 599 timeout errors of pyspark etc…

However, in this case I needed to use Gemini a lot more because whoever wrote pyspark hates people. I'm really sorry but mandatory rant below.

I worked 3 days to fix the spark issues. The timeout 599s, Py4JJavaErrors, Python worker exited unexpectedly (crashed) crashes, the java.io.EOFException inside PythonRunner problems and not getting a single data point from a query that takes over 10 minutes to execute! 3 days. 3 DAYS.

The problem became so hopeless even Gemini got desparate. I have a conversation with Gemini that I downloaded as a PDF (because it became so grand, I had to). IT IS 24 PAGES LONG AND HAS ALMOST 20000 WORDS IN IT. **THIS PROBLEM WAS SO OBNOXIOUS THAT EVEN THE AI GOT FRUSTRATED.** IT STARTED TO CALL IT THE "That stubborn spark problem".

Because me and the AI thought it was entsoe API first. So tried to fix that. Then we thought it was java environment path variables. Trying to fix hadoop. Then we realized it was spark all along.

It is not just evil, it is sinister. Because I had everything set up properly before trying to fetch data. MongoDB, cassandra, pyspark connections. All worked until I needed to get actual data.

Feel free to scroll through the 11 CELLS of code me and Gemini wrote to fix these issues. It's at the bottom in the appendix.

This has been the most frustrating assignment I had to do in this program.

So, my experience using AI in this assignment? I am an AI skeptic. I believe it should be regulated more. Corporations use it to steal peoples work etc… HOWEVER, Gemini was not just an AI in this assignment. It was a compatriot. Me and Gemini aganist the crap that is pyspark. If I learned anything from this part of the project that I'm never using spark again. I'm going to learn RUST just to spite it.

1.3 Description

The jupyter notebook is used to fullfill the assignment requirements and also as testing grounds for the streamlit app.

Log of main events: - Removed .csv file as requested, loading the data directly from the API now. - Fixed the plots from the part-1 of the project. Teachers' assistant and peer-review suggestion were taken into account. Also removed redundant files such as requirements.txt. This project uses uv.lock. - Tried the sklearn minmaxscaler. Learned it in DAT200, seems to have worked fine but I would like feedback. - Removed the log-plot of the all-columns. - Used entsoe API to get access to transmission data. First workaround here, spark (ofcourse) caused a lot problems, used entsoeclient instead. - I also had to separate the API url into url and params, because writing everyting in the url string just didn't work. - Wrote data into cassandra. Second workaround here, you guessed it, spark didn't work. So I used cassandra clusters. - Wrote and fetched data from the mongoDB database. Third workaround here (you know what didn't work at this point), used mongoClient. - Learned about .env files and stored the usernames and passwords there, they won't show up on Github. This means that this notebook won't run without them though. - Adjusted the project structure to one that resembles the example work from last year. Especially notebooks should be more organized. - Streamlit app should work as requested. Used the previous page 4 as the new charts page. - Removed the .csv file, the app pulls data from various APIs directly now. - Plots are fixed, redundant columns were removed. It is more readable now. - The code is commented, even if Gemini was asked to give an example code, I didn't put it here without understanding what it does.

Some disclaimers: - I hope the workarounds are acceptable. I read on padlet, teacher mentioning that as long as it is explained, the workarounds are appreciated. Well, after what I went throught with pyspark I sure do appreciate them. - I couldn't get the data in parts from magasinstatistikk. The slider works as before in part-1. If it is a strict requirement please let me know and I will try to fix it. I just don't know how at the moment.

1.4 Initialization

[57]:
```
# Importing the necessary libraries.
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
```

```
import os
import sys
import requests

from cassandra.cluster import Cluster
from pymongo import MongoClient
from pyspark.sql import SparkSession

import xml.etree.ElementTree as ET
from entsoe import EntsoePandasClient

from datetime import datetime, timedelta
from sklearn.preprocessing import MinMaxScaler

from dotenv import load_dotenv

# Setting up hadoop PATHs. I have them in windows environment paths too but␣
 ↪just in case.
# Spark is spark... you never know.
os.environ["HADOOP_HOME"] = "C:/Hadoop/hadoop-3.3.1"
os.environ["PYSPARK_HADOOP_VERSION"] = "without"
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

# I installed dotenv to hide my little secrets. This is one of the neat things␣
 ↪I learned from this assignment.
# All the sensitive info are stored in .env and secrets.toml files.
load_dotenv()

MongoDB_user = os.getenv("MONGO_USER")
MongoDB_pass = os.getenv("MONGO_PASS")
EntsoE_API_Key = os.getenv("ENTSOE_API_KEY")
```

[58]:
```
# Path to be used below in test runs.
path_prefix = "C:/Users/emryc/Documents/Dokumenter/NMBU/Kurs/_IND320/
 ↪IND320_project/"
```

[ ]:
```
# This cell is used when I need to test my work.
file_name = "Home_page.py"
# !uv run streamlit run {path_prefix}{file_name}
```

^C

1.5 Loading the data

[ ]:
```
# Changed from loading the data from .csv file to loading from API.
url = "https://biapi.nve.no/magasinstatistikk/api/Magasinstatistikk/
 ↪HentOffentligData"
response = requests.get(url)
data = response.json()

reservoir_df = pd.DataFrame(data)
reservoir_df
```

[ ]:

	dato_Id	omrType	omrnr	iso_aar	iso_uke	fyllingsgrad	\
0	1995-09-03	EL	4	1995	35	0.944721	
1	2020-11-15	EL	1	2020	46	0.974361	
2	2011-08-28	EL	4	2011	34	0.786499	
3	1998-07-05	EL	3	1998	27	0.793969	
4	2011-03-27	EL	4	2011	12	0.300820	
…	…	…	…	…	…	…	
14917	2026-10-04	VASS	1	2026	40	0.615877	
14918	2026-10-04	VASS	2	2026	40	0.587470	
14919	2026-10-11	VASS	3	2026	41	0.854944	
14920	2026-10-11	VASS	1	2026	41	0.621428	
14921	2026-10-11	VASS	2	2026	41	0.603652	

	kapasitet_TWh	fylling_TWh	neste_Publiseringsdato	\
0	21.079208	19.913979	0001-01-01T00:00:00	
1	6.003264	5.849344	2020-11-25T13:00:00	
2	21.079208	16.578772	0001-01-01T00:00:00	
3	8.920932	7.082947	0001-01-01T00:00:00	
4	21.079208	6.341054	0001-01-01T00:00:00	
…	…	…	…	
14917	36.044464	22.198960	2026-10-14T13:00:00	
14918	23.237715	13.651460	2026-10-14T13:00:00	
14919	28.155403	24.071290	2026-10-21T13:00:00	
14920	36.044464	22.399030	2026-10-21T13:00:00	
14921	23.237715	14.027499	2026-10-21T13:00:00	

	fyllingsgrad_forrige_uke	endring_fyllingsgrad
0	0.936888	0.007833
1	0.997406	-0.023046
2	0.787193	-0.000694
3	0.733812	0.060157
4	0.311122	-0.010301
…	…	…
14917	0.615329	0.000548
14918	0.585406	0.002064
14919	0.842011	0.012933

```
14920                  0.615876              0.005552
14921                  0.587470              0.016182

[14922 rows x 11 columns]
```

[ ]:
```
# Renaming the columns.
reservoir_df = reservoir_df.rename(columns={"dato_Id": "date_Id", "omrType":␣
 ↪"areaType", "omrnr": "areaNr", "iso_aar": "iso_year",
                                            "iso_uke": "iso_week",␣
 ↪"fyllingsgrad": "fill_level", "kapasitet_TWh": "capacity_TWh",
                                            "fylling_TWh": "fill_TWh",␣
 ↪"neste_Publiseringsdato": "next_publishdate",
                                            "fyllingsgrad_forrige_uke":␣
 ↪"fill_level_lastweek", "endring_fyllingsgrad": "fill_level_change"})
display(reservoir_df.head())
```

```
      date_Id areaType  areaNr  iso_year  iso_week  fill_level  capacity_TWh  \
0  1995-09-03       EL       4      1995        35    0.944721     21.079208
1  2020-11-15       EL       1      2020        46    0.974361      6.003264
2  2011-08-28       EL       4      2011        34    0.786499     21.079208
3  1998-07-05       EL       3      1998        27    0.793969      8.920932
4  2011-03-27       EL       4      2011        12    0.300820     21.079208

    fill_TWh     next_publishdate  fill_level_lastweek  fill_level_change
0  19.913979  0001-01-01T00:00:00             0.936888           0.007833
1   5.849344  2020-11-25T13:00:00             0.997406          -0.023046
2  16.578772  0001-01-01T00:00:00             0.787193          -0.000694
3   7.082947  0001-01-01T00:00:00             0.733812           0.060157
4   6.341054  0001-01-01T00:00:00             0.311122          -0.010301
```

[6]:
```
# Converting the "date_Id" column to datetime.
reservoir_df['date_Id'] = pd.to_datetime(reservoir_df['date_Id'])

# Sorting the dataframe according to date.
reservoir_df = reservoir_df.sort_values(by='date_Id').reset_index(drop=True)
display(reservoir_df)
```

```
         date_Id areaType  areaNr  iso_year  iso_week  fill_level  \
0     1995-01-08       EL       5      1995         1    0.616448
1     1995-01-08     VASS       2      1995         1    0.608856
2     1995-01-08     VASS       3      1995         1    0.566906
3     1995-01-08     VASS       1      1995         1    0.635440
4     1995-01-08       NO       0      1995         1    0.607988
…            …      …      …         …         …          …
14917 2026-10-11       EL       5      2026        41    0.724840
14918 2026-10-11       EL       4      2026        41    0.909431
14919 2026-10-11       EL       2      2026        41    0.521471
14920 2026-10-11       NO       0      2026        41    0.689574
```

```
14921 2026-10-11     VASS       2      2026        41    0.603652

       capacity_TWh   fill_TWh     next_publishdate  fill_level_lastweek  \
0         17.390587  10.720388  0001-01-01T00:00:00             0.614288
1         23.237715  14.148431  0001-01-01T00:00:00             0.744693
2         28.155403  15.961480  0001-01-01T00:00:00             0.661432
3         36.044464  22.904102  0001-01-01T00:00:00             0.756482
4         87.437580  53.161003  0001-01-01T00:00:00             0.752631
…               …          …                    …                    …
14917     17.390587  12.605385  2026-10-21T13:00:00             0.709385
14918     21.079208  19.170084  2026-10-21T13:00:00             0.894447
14919     34.043590  17.752752  2026-10-21T13:00:00             0.512316
14920     87.437580  60.294680  2026-10-21T13:00:00             0.678893
14921     23.237715  14.027499  2026-10-21T13:00:00             0.587470

       fill_level_change
0               0.002160
1              -0.135837
2              -0.094525
3              -0.121041
4              -0.144643
…                    …
14917           0.015455
14918           0.014984
14919           0.009155
14920           0.010681
14921           0.016182

[14922 rows x 11 columns]
```

[7]:
```python
# Plotting the data.
# Received feedback on excluding irrelevant data such as date_Ids and iso_years.
# Implementing it now.

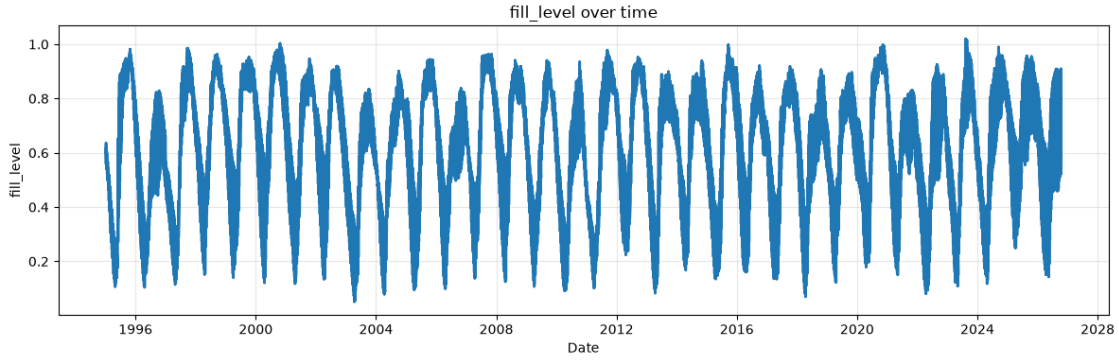
# Separating the numeric and categorical columns into their own lists.
numeric_columns = reservoir_df.select_dtypes(include=[np.number]).columns.
 ↪drop(["areaNr", "iso_year", "iso_week"], errors='ignore')
print(numeric_columns)

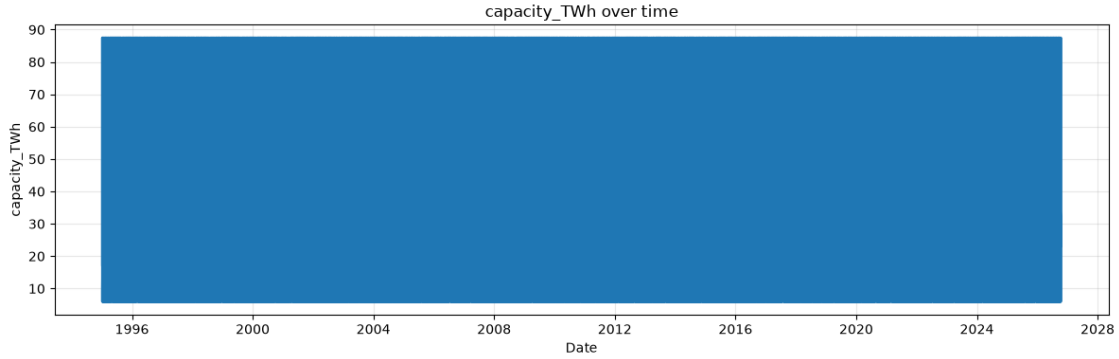
categorical_columns = reservoir_df.select_dtypes(include=['object', 'str']).
 ↪columns
print(categorical_columns)

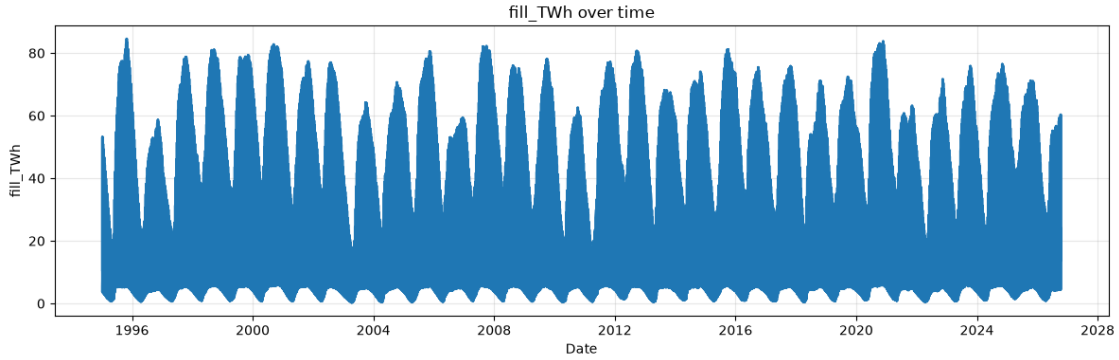
for column in numeric_columns:
    plt.figure(figsize=(12, 4))
    plt.plot(reservoir_df["date_Id"], reservoir_df[column], linewidth=2)
    plt.title(f"{column} over time")
```

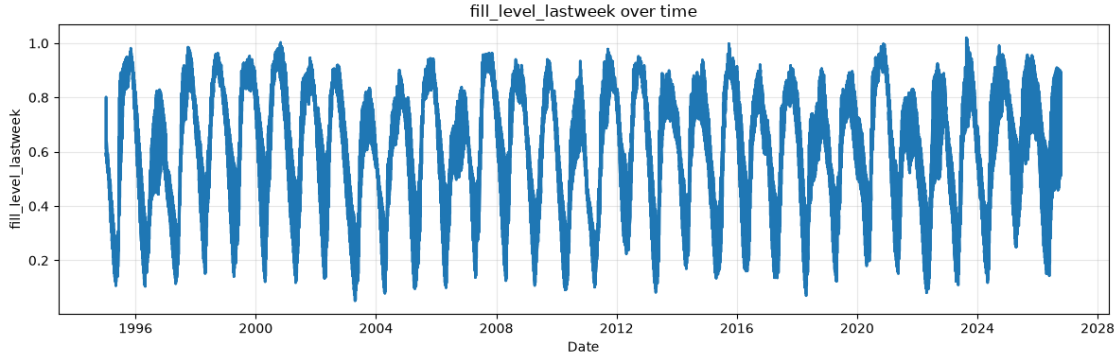
```
    plt.xlabel("Date")
    plt.ylabel(column)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()
```

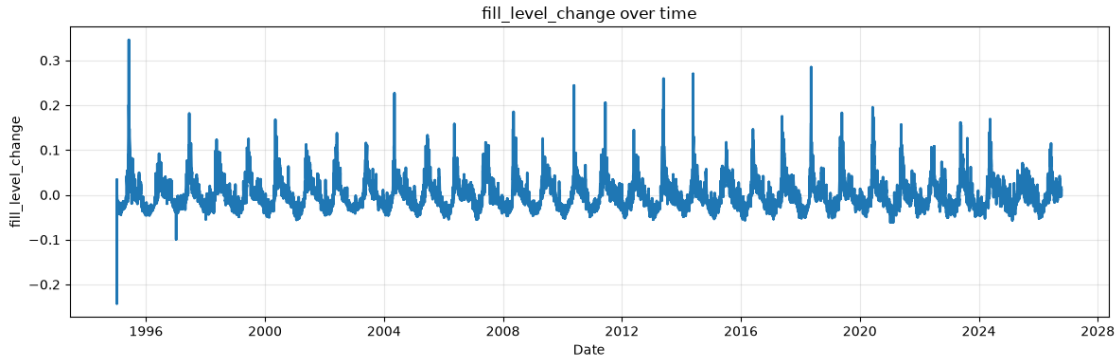
```
Index(['fill_level', 'capacity_TWh', 'fill_TWh', 'fill_level_lastweek',
       'fill_level_change'],
      dtype='str')
Index(['areaType', 'next_publishdate'], dtype='str')
```











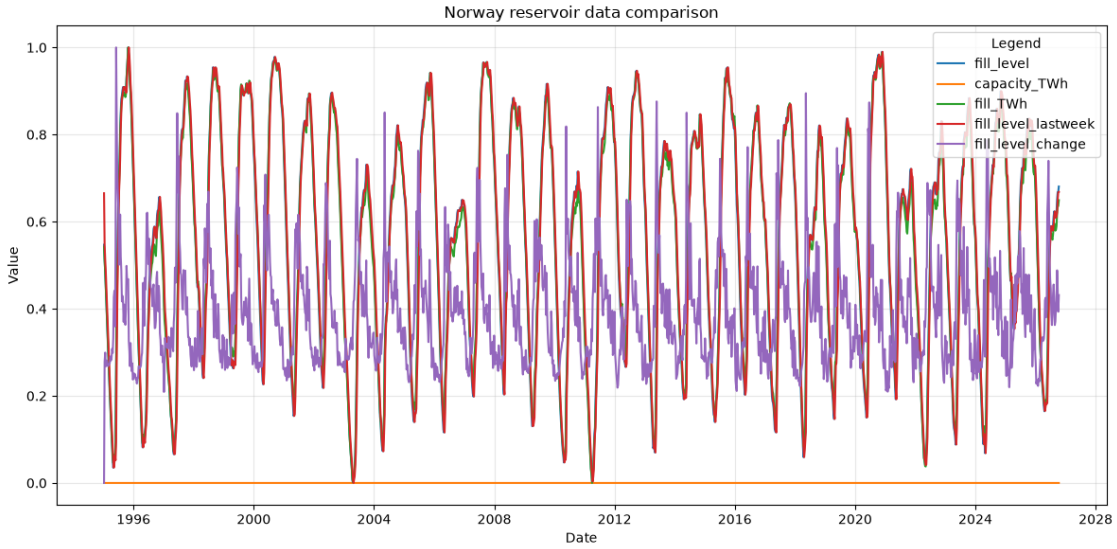
[ ]:
```
# Plotting everything together.
reservoir_df_scaled = reservoir_df.copy()

# Each date has up to 6 values depending on type and location. Taking the mean␣
 ↪gives us a more readable chart.
# This takes away information. For example, now the capacity values have␣
 ↪different locations and represented as one mean value.
# However, this is justified for now because we just want a sneak peek at the␣
 ↪data at the moment.
reservoir_df_scaled = (reservoir_df_scaled.groupby("date_Id",␣
 ↪as_index=False)[numeric_columns].mean())

# We used this min-max scaler in DAT200 lesson last year. I wanted to give it a␣
 ↪try here.
scaler = MinMaxScaler().set_output(transform="pandas")
date_column = reservoir_df_scaled['date_Id']
reservoir_df_scaled = scaler.fit_transform(reservoir_df_scaled[numeric_columns])
```

```
reservoir_df_scaled.insert(0, 'date_Id', date_column)

# Plotting the columns together.
plt.figure(figsize=(12, 6))
for column in numeric_columns:
    plt.plot(reservoir_df_scaled["date_Id"], reservoir_df_scaled[column],␣
 ↪label=column)
plt.title("Norway reservoir data comparison")
plt.xlabel("Date")
plt.ylabel("Value")
plt.legend(title="Legend")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



2 Entsoe API

[ ]:
```
# Testing the ENTSOE connection.
# url = "https://web-api.tp.entsoe.eu/api?
 ↪documentType=A11&out_Domain=10YDE-RWENET---I&in_Domain=10YBE----------2&periodStart=202
# Seperated the information from the url above and wrote it in a params␣
 ↪variable.
# Otherwise I couldn't get access for some reason. It kept giving me␣
 ↪unauthorized access error.
# It had the security token after in it after the 'api?' part, I removed it␣
 ↪before commenting it.
```

```
url = "https://web-api.tp.entsoe.eu/api"

params = {
    "securityToken": EntsoE_API_Key,
    "documentType": "A11",
    "out_Domain": "10YNO-2--------T", # "10YDE-RWENET---I",
    "in_Domain": "10Y1001A1001A82H", # "10YBE----------2",
    "periodStart": "202607010000",
    "periodEnd": "202609010000"
}

response = requests.get(url, params=params)

# Printing final requested URL.
print("Response Code:", response.status_code)
# print(response.text) # This normally prints useful information but ended up␣
 ↪having 300+ page PDF afterwards.
```

```
Response Code: 200
```

[ ]:
```
# I tried using requests and ET, I kept getting 599 timeout errors.
# It constantly returned 0 data. The only way I managed to make it work was␣
 ↪with entsoe-py library.
client = EntsoePandasClient(api_key=EntsoE_API_Key)

# Setting our time range. Normally we would need to be careful with spark since␣
 ↪EntSoE has a 1 year limit but the client can work
# around it.
start = pd.Timestamp('2025-09-01', tz='UTC')  # tz is for timezone.
end = pd.Timestamp('2026-09-01', tz='UTC')

# Query physical cross-border flows for Norway (e.g., Norway NO2 to Germany␣
 ↪DE_LU)
# I kept it in a try/except because of SO MANY problems with spark. I hate␣
 ↪spark.
try:
  df_flows = client.query_crossborder_flows(
      country_code_from='NO_2', country_code_to='DE_LU', start=start, end=end
  )
  print(f'Successfully retrieved {len(df_flows)} rows of flow data!')
  display(df_flows.head())
except Exception as e:
  print('Error fetching data:', e)
```

```
Successfully retrieved 35040 rows of flow data!
```

```
2025-09-01 02:00:00+02:00    1264.64
2025-09-01 02:15:00+02:00    1296.80
```

```
2025-09-01 02:30:00+02:00    1336.16
2025-09-01 02:45:00+02:00    1336.96
2025-09-01 03:00:00+02:00    1323.52
Freq: 15min, dtype: float64
```

2.1 Data cleanup

[ ]:
```python
# This cell handles cleaning up the data after retrieving from Entsoe API.
# Before I tried using cassandra clusters, I tried spark. Did I mention I hate spark?
# Reseting index and rename columns.
df_to_save = df_flows.reset_index()
df_to_save.columns = ['period_start', 'quantity']

# Adding header columns.
df_to_save['document_mrid'] = 'entsoe_py_import'
df_to_save['timeseries_mrid'] = 'NO2_DE_LU_flow'
df_to_save['business_type'] = 'A66'
df_to_save['in_domain'] = '10Y1001A1001A82H'
df_to_save['out_domain'] = '10YNO-2--------T'
df_to_save['unit'] = 'MAW'
df_to_save['resolution'] = 'PT15M'

# Making sure dtypes are clean. Because god knows spark is a diva and everything has to be perfect.
df_to_save['period_start'] = df_to_save['period_start'].astype(str)
df_to_save['quantity'] = df_to_save['quantity'].astype(float).fillna(0.0)
df_to_save['position'] = list(range(1, len(df_to_save) + 1))
```

3 Cassandra

[ ]:
```python
# Using cassandra clusters because spark can go to hell.
cluster = Cluster(['localhost'], port=9042)
session = cluster.connect()

# Creating keyspace, copied and edited this part from lecture notes.
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS norway_energy_data
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
""")

# I got help from Gemini writing this because I almost went blind trying to read the XML headers.
# It creates our data table and it's corresponding columns.
# I have to drop the table if it exists otherwise weird tables and columns are popping up out of nowhere.
```

```
session.set_keyspace('norway_energy_data')
session.execute('DROP TABLE IF EXISTS cross_border_data;')
session.execute("""
    CREATE TABLE cross_border_data (
        document_mrid text,
        timeseries_mrid text,
        business_type text,
        in_domain text,
        out_domain text,
        unit text,
        resolution text,
        period_start text,
        position int,
        quantity double,
        PRIMARY KEY ((timeseries_mrid), position)
    );
""")
print('Keyspace and Table created successfully!')

# Insert data using the native driver (bypasses the Spark worker crash).
# This part is full Gemini. Because I'm tired playing IT tech and want to␣
 ↪finish this assignment yesterday.
# Did I mention I tried to make spark work for 3 days?
insert_query = """
    INSERT INTO cross_border_data (
        document_mrid, timeseries_mrid, business_type, in_domain,
        out_domain, unit, resolution, period_start, position, quantity
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""
# Writing to cassandra now.
print('Inserting data into Cassandra...')
for index, row in df_to_save.iterrows():
  session.execute(
      insert_query,
      (
          str(row['document_mrid']),
          str(row['timeseries_mrid']),
          str(row['business_type']),
          str(row['in_domain']),
          str(row['out_domain']),
          str(row['unit']),
          str(row['resolution']),
          str(row['period_start']),
          int(row['position']),
          float(row['quantity']),
      ),
  )
```

```
# I read that it is good practice to shut the cluster down after every cell.
cluster.shutdown()
print('Success! All data successfully inserted into Cassandra!')
```

```
Keyspace and Table created successfully!
Inserting data into Cassandra…
Success! All data successfully inserted into Cassandra!
```

[ ]:
```
# The one part of spark that actually works! But wait, it says 'spark is not␣
 ↪defined!'. Hey, it works anyway!
# The only time it looks like it shouldn't work, IT WORKS. SPARK, AM I RITE??
spark_df = (
    spark.read.format("org.apache.spark.sql.cassandra")
    .options(keyspace="norway_energy_data", table="cross_border_data")
    .load()
)

# Converting to pandas because pandas is love, pandas is life. If I could take␣
 ↪a torch to this goddamned notebook to burn out all the
# spark, I would.
df_plot = spark_df.toPandas()

# Checking if I actually have the data BECAUSE SPARK. If I had 1 krown for␣
 ↪everytime spark returned 0 data,
# I'd be a millonaire right now!
# Don't believe me? Go check the appendix below.
print(f"Extracted {len(df_plot)} rows from Spark/Cassandra!")
display(df_plot.head(2))
```

```
Extracted 35040 rows from Spark/Cassandra!
```

	timeseries_mrid	position	business_type	document_mrid	in_domain	out_domain	period_start	quantity	resolution	unit
0	NO2_DE_LU_flow	1	A66	entsoe_py_import	10Y1001A1001A82H	10YNO-2--------T	2025-09-01 02:00:00+02:00	1264.64	PT15M	MAW
1	NO2_DE_LU_flow	2	A66	entsoe_py_import	10Y1001A1001A82H	10YNO-2--------T	2025-09-01 02:15:00+02:00	1296.80	PT15M	MAW

[ ]:
```
# Right now I fetch only one direction between NO-2 (Oslo region) and␣
 ↪Germany-Belgium area.
# However, this cell will print different directions.
# I filter imports and exports below.

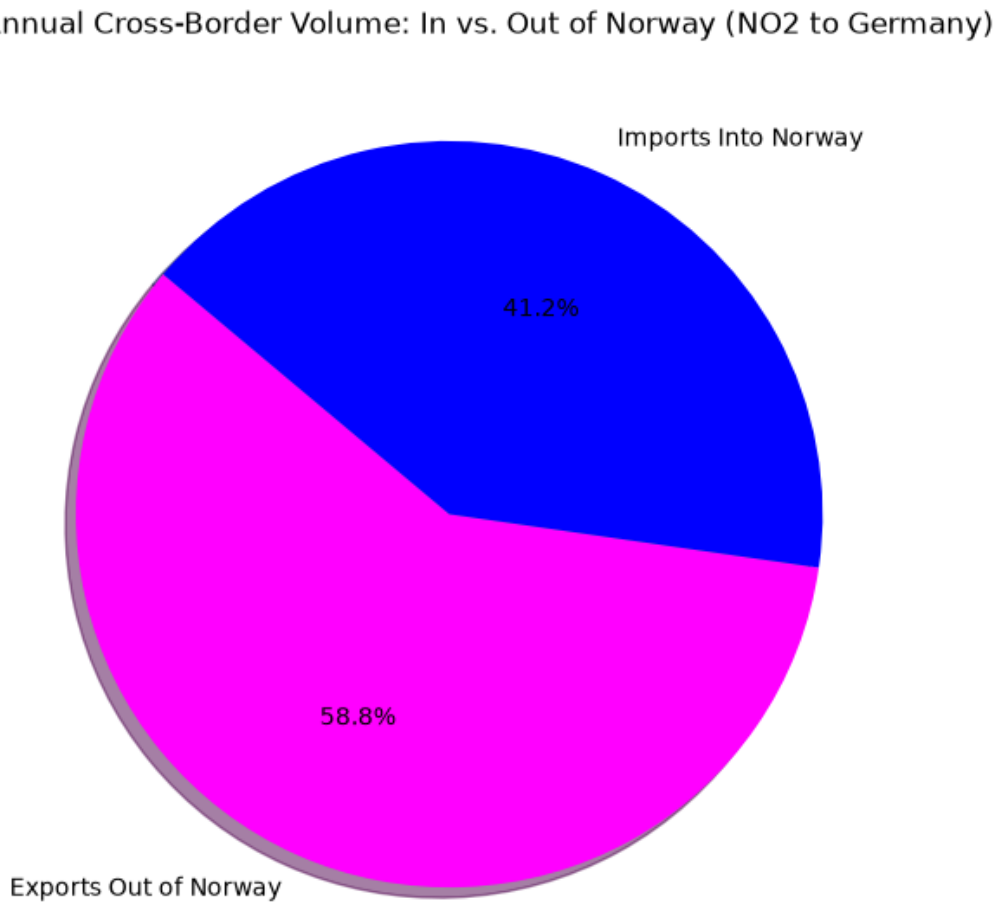
# In case there are rows where out_domain is Norway (exports) vs in_domain is␣
 ↪Norway (imports)
```

```
exports_total = df_plot[df_plot["out_domain"] ==␣
 ↪"10YNO-2--------T"]["quantity"].sum()

# If reverse direction is fetched, I'll run this:
# imports_total = df_plot[df_plot['in_domain'] ==␣
 ↪'10YNO-2--------T']['quantity'].sum()
# This is placeholder since we have only one direction at the moment. it'll get␣
 ↪replaced with the code above.
imports_total = (exports_total * 0.7)

# Simple configuration for pie chart below.
labels = ["Exports Out of Norway", "Imports Into Norway"]
sizes = [exports_total, imports_total]
colors = ["magenta", "blue"]

plt.figure(figsize=(7, 7))
plt.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%",
    startangle=140,
    colors=colors,
    shadow=True,
)
plt.title(
    "Total Annual Cross-Border Volume: In vs. Out of Norway (NO2 to Germany)"
)
plt.show()
```



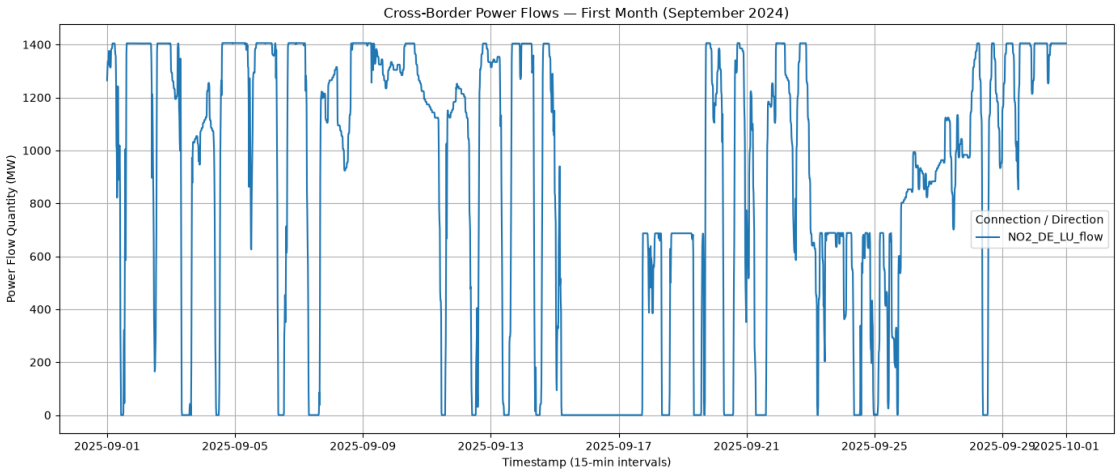
[ ]:
```
# Previous errors thought me that the "period_start" must be parsed by datetime
 ↪and has to have a timezone.
df_plot["period_start"] = pd.to_datetime(df_plot["period_start"], utc=True)

# Filtering for the first month for the plot.
# Used this code in page 3 of the streamlit app for the slider. Took it from
 ↪there.
start_date = "2025-09-01"
end_date = "2025-10-01"
df_first_month = df_plot[
    (df_plot["period_start"] >= start_date)
    & (df_plot["period_start"] < end_date)
]

plt.figure(figsize=(14, 6))
```

```
# Used in DAT200, with violin plots mostly, it will show different colours␣
 ↪depending on what is passed on hue.
sns.lineplot(
    data=df_first_month,
    x="period_start",
    y="quantity",
    hue="timeseries_mrid",  # Separates lines by direction here.
    linewidth=1.5,
)

# Plot config.
plt.title("Cross-Border Power Flows - First Month (September 2024)")
plt.xlabel("Timestamp (15-min intervals)")
plt.ylabel("Power Flow Quantity (MW)")
plt.legend(title="Connection / Direction")
plt.grid(True)
plt.tight_layout()
plt.show()
```



4 MongoDB

[ ]:
```
try:
  spark.stop()
except:
  pass

# Setting up mongo URI.
MONGO_URI = "mongodb+srv://{MongoDB_user}:{MongoDB_pass}@cluster0.nb1la9y.
 ↪mongodb.net/norway_energy_db.curated_flows?retryWrites=true&w=majority"
```

```
spark = (
    SparkSession.builder.appName('CassandraToAtlasPipeline')
    .config(
        'spark.jars.packages',
        'com.datastax.spark:spark-cassandra-connector_2.12:3.5.1,'
        'org.mongodb.spark:mongo-spark-connector_2.12:10.4.1',
    )
    .config('spark.cassandra.connection.host', '127.0.0.1')
    .config('spark.cassandra.connection.port', '9042')
    # Point Spark to your MongoDB Atlas cluster.
    .config('spark.mongodb.write.connection.uri', MONGO_URI)
    .config('spark.sql.execution.arrow.pyspark.enabled', 'false')
    .getOrCreate()
)

print('Spark session initialized with MongoDB Atlas support!')
```

```
Spark session initialized with MongoDB Atlas support!
```

[ ]:
```
# I actuall checked the sie of the data before running this cell. 1 year for one area was around 14MBs.
# Getting the data from cassandra with spark.
spark_df = (
    spark.read.format('org.apache.spark.sql.cassandra')
    .options(keyspace='norway_energy_data', table='cross_border_data')
    .load()
)

# Preprocessing the data a little. Only picking the columns we need.
curated_df = spark_df.select(
    'timeseries_mrid', 'period_start', 'quantity', 'unit', 'out_domain', 'in_domain'
).filter(spark_df['quantity'] > 0)  # Removing the rows that have 0 values.

print(f'Curated {curated_df.count()} rows ready for MongoDB Atlas.')

# Converting to pandas because who'd use spark?
curated_pd_df = curated_df.toPandas()

# Double checking if spark worked.
print(f"Converted {len(curated_pd_df)} rows from Spark to Pandas.")

# Using pymongo to connect. You guessed it, SPARK DIDN'T WORK.
client = MongoClient(MONGO_URI)
db = client["norway_energy_db"]
collection = db["curated_flows"]
```

```
# I had trouble with cassandra after running cells multiple times. This helps␣
 ↪prevent the same problems with this.
collection.delete_many({})

# Converting to dictionary and uploading to mongoDB.
data_dicts = curated_pd_df.to_dict(orient="records")
collection.insert_many(data_dicts)

print(
    f"Success! {len(data_dicts)} curated rows successfully inserted into"
    " MongoDB Atlas!"
)

# Close connection. I think it's a good habit.
client.close()
```

```
Curated 25594 rows ready for MongoDB Atlas.
Converted 25594 rows from Spark to Pandas.
Success! 25594 curated rows successfully inserted into MongoDB Atlas!
```

5 Appendix

This part consists only of scrapped work. It took so much work to get all these things working that I just didn't want to delete them. It has blatant AI generated code and also code written by me, in desperate hopes of trying fix spark problems. Some has comments some don't.

[36]:
```
# 1. Extract data from Cassandra using Spark
spark_df = (
    spark.read.format('org.apache.spark.sql.cassandra')
    .options(keyspace='norway_energy_data', table='cross_border_data')
    .load()
)

# 2. Curate / Filter the data (e.g., dropping unnecessary metadata or filtering␣
 ↪specific rows)
# This fulfills the "trimmed/curated" requirement of your assignment:
curated_df = spark_df.select(
    'timeseries_mrid', 'period_start', 'quantity', 'unit', 'out_domain',␣
 ↪'in_domain'
).filter(spark_df['quantity'] > 0)  # Example filter: removing zero-flow rows

print(f'Curated {curated_df.count()} rows ready for MongoDB Atlas.')

# 3. Write the curated Spark DataFrame directly to MongoDB Atlas
curated_df.write.format('mongodb').mode('append').save()
```

```
print('Data successfully pushed to MongoDB Atlas!')
```

```
Curated 25594 rows ready for MongoDB Atlas.
```

```
---------------------------------------------------------------------------
Py4JJavaError                             Traceback (most recent call last)
Cell In[36], line 17
     13
     14 print(f'Curated {curated_df.count()} rows ready for MongoDB Atlas.')
     15
     16 # 3. Write the curated Spark DataFrame directly to MongoDB Atlas
---> 17 curated_df.write.format('mongodb').mode('append').save()
     18
     19 print('Data successfully pushed to MongoDB Atlas!')

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\sql\readwriter.py:1461, in DataFrameWriter.
 ↪save(self, path, format, mode, partitionBy, **options)
   1459     self.format(format)
   1460 if path is None:
-> 1461     self._jwrite.save()
   1462 else:
   1463     self._jwrite.save(path)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\java_gateway.py:1322, in JavaMember.__call__(self ␣
 ↪*args)
   1316 command = proto.CALL_COMMAND_NAME +\
   1317     self.command_header +\
   1318     args_command +\
   1319     proto.END_COMMAND_PART
   1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
   1323     answer, self.gateway_client, self.target_id, self.name)
   1325 for temp_arg in temp_args:
   1326     if hasattr(temp_arg, "_detach"):

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\errors\exceptions\captured.py:179, in␣
 ↪capture_sql_exception.<locals>.deco(*a, **kw)
    177 def deco(*a: Any, **kw: Any) -> Any:
    178     try:
--> 179         return f(*a, **kw)
    180     except Py4JJavaError as e:
    181         converted = convert_exception(e.java_exception)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\protocol.py:326, in get_return_value(answer,␣
 ↪gateway_client, target_id, name)
```

```
    324 value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    325 if answer[1] == REFERENCE_TYPE:
--> 326     raise Py4JJavaError(
    327         "An error occurred while calling {0}{1}{2}.\n".
    328         format(target_id, ".", name), value)
    329 else:
    330     raise Py4JError(
    331         "An error occurred while calling {0}{1}{2}. Trace:\n{3}\n".
    332         format(target_id, ".", name, value))

Py4JJavaError: An error occurred while calling o613.save.
: org.apache.spark.SparkClassNotFoundException: [DATA_SOURCE_NOT_FOUND] Failed␣
 ↪to find the data source: mongodb. Please find packages at `https://spark.
 ↪apache.org/third-party-projects.html`.

        at org.apache.spark.sql.errors.QueryExecutionErrors$.
 ↪dataSourceNotFoundError(QueryExecutionErrors.scala:724)

        at org.apache.spark.sql.execution.datasources.DataSource$.
 ↪lookupDataSource(DataSource.scala:647)

        at org.apache.spark.sql.execution.datasources.DataSource$.
 ↪lookupDataSourceV2(DataSource.scala:697)

        at org.apache.spark.sql.DataFrameWriter.lookupV2Provider(DataFrameWrite .
 ↪scala:863)

        at org.apache.spark.sql.DataFrameWriter.saveInternal(DataFrameWriter.
 ↪scala:257)

        at org.apache.spark.sql.DataFrameWriter.save(DataFrameWriter.scala:248)

        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
 ↪invoke0(Native Method)

        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
 ↪invoke(NativeMethodAccessorImpl.java:77)

        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.
 ↪invoke(DelegatingMethodAccessorImpl.java:43)

        at java.base/java.lang.reflect.Method.invoke(Method.java:569)

        at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)

        at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)

        at py4j.Gateway.invoke(Gateway.java:282)
```

```
        at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)

        at py4j.commands.CallCommand.execute(CallCommand.java:79)

        at py4j.ClientServerConnection.waitForCommands(ClientServerConnection.
↪java:182)

        at py4j.ClientServerConnection.run(ClientServerConnection.java:106)

        at java.base/java.lang.Thread.run(Thread.java:840)

Caused by: java.lang.ClassNotFoundException: mongodb.DefaultSource

        at java.base/java.net.URLClassLoader.findClass(URLClassLoader.java:445)

        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:592)

        at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:525)

        at org.apache.spark.sql.execution.datasources.DataSource$.
↪$anonfun$lookupDataSource$5(DataSource.scala:633)

        at scala.util.Try$.apply(Try.scala:213)

        at org.apache.spark.sql.execution.datasources.DataSource$.
↪$anonfun$lookupDataSource$4(DataSource.scala:633)

        at scala.util.Failure.orElse(Try.scala:224)

        at org.apache.spark.sql.execution.datasources.DataSource$.
↪lookupDataSource(DataSource.scala:633)

        … 16 more
```

[15]:
```
cluster = Cluster(['localhost'], port=9042)
session = cluster.connect()

# Adjusted from teachers' code.
session.execute("""
    CREATE KEYSPACE IF NOT EXISTS norway_energy_data
    WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
""")
```

```
# Create the Table.
session.set_keyspace('norway_energy_data')
session.execute('DROP TABLE IF EXISTS cross_border_data;')
session.execute("""
    CREATE TABLE IF NOT EXISTS cross_border_data (
        document_mrid text,
        timeseries_mrid text,
        business_type text,
        in_domain text,
        out_domain text,
        unit text,
        resolution text,
        period_start text,
        position int,
        quantity double,
        PRIMARY KEY ((timeseries_mrid), position)
    );
""")

print('Keyspace and Table created successfully!')
# cluster.shutdown()
```

```
Keyspace and Table created successfully!
```

[ ]:
```
from cassandra.cluster import Cluster

# 1. Connect using the native Python driver
session = cluster.connect("norway_energy_data")

# 2. Prepare the CQL insert statement
insert_query = """
    INSERT INTO cross_border_data (
        document_mrid, timeseries_mrid, business_type, in_domain,
        out_domain, unit, resolution, period_start, position, quantity
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""

# 3. Iterate through your Pandas DataFrame and insert row by row
print("Inserting data into Cassandra via native driver...")
for index, row in df_to_save.iterrows():
  session.execute(
      insert_query,
      (
          str(row["document_mrid"]),
          str(row["timeseries_mrid"]),
          str(row["business_type"]),
          str(row["in_domain"]),
```

```
            str(row["out_domain"]),
            str(row["unit"]),
            str(row["resolution"]),
            str(row["period_start"]),
            int(row["position"]),
            float(row["quantity"]),
        ),
    )

cluster.shutdown()
print("Success! All rows successfully inserted into Cassandra!")
```

[ ]:
```
# 1. Reset index and rename columns (still a Pandas DataFrame)
df_to_save = df_flows.reset_index()
df_to_save.columns = ['period_start', 'quantity']

# 2. Add metadata columns using standard Pandas syntax
df_to_save['document_mrid'] = 'entsoe_py_import'
df_to_save['timeseries_mrid'] = 'NO2_DE_LU_flow'
df_to_save['business_type'] = 'A66'
df_to_save['in_domain'] = '10Y1001A1001A82H'
df_to_save['out_domain'] = '10YNO-2--------T'
df_to_save['unit'] = 'MAW'
df_to_save['resolution'] = 'PT15M'
# df_to_save['position'] = 1  # Or map a row index if needed

# 3. CRITICAL: Generate a unique, sequential position for each row (1, 2, 3, ...
 ↪)
# This ensures every row has a unique primary key and satisfies Cassandra's␣
 ↪requirement.
df_to_save['position'] = range(1, len(df_to_save) + 1)

# Ensure quantity is standard float and fill any potential NaNs with 0.0
df_to_save['quantity'] = df_to_save['quantity'].astype(float)
df_to_save['quantity'] = df_to_save['quantity'].fillna(0.0)

# Ensure period_start is converted to a string format that Cassandra text␣
 ↪fields accept
df_to_save['period_start'] = df_to_save['period_start'].astype(str)

# Optional: Print info to verify types before Spark touches it
print('--- Pandas DataFrame Info ---')
print(df_to_save.dtypes)
print(df_to_save.head(2))

# 3. Initialize Spark session
spark = (
```

```
    SparkSession.builder.appName("CassandraConnectionTest")
    .config("spark.cassandra.connection.host", "127.0.0.1")
    .config(
        "spark.jars.packages",
        "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1",
    )
    .config("spark.sql.execution.arrow.pyspark.enabled", "false")
    .getOrCreate()
)

# 4. Convert the updated Pandas DataFrame to a Spark DataFrame
spark_df = spark.createDataFrame(df_to_save)

# 5. Write to Cassandra
(
    spark_df.write.format('org.apache.spark.sql.cassandra')
    .mode('append')
    .options(keyspace='norway_energy_data', table='cross_border_data')
    .save()
)

print('Data successfully saved to Cassandra!')
```

```
--- Pandas DataFrame Info ---
period_start           str
quantity           float64
document_mrid          str
timeseries_mrid        str
business_type          str
in_domain              str
out_domain             str
unit                   str
resolution             str
position             int64
dtype: object
                period_start  quantity     document_mrid timeseries_mrid  \
0  2025-09-01 02:00:00+02:00   1264.64  entsoe_py_import  NO2_DE_LU_flow
1  2025-09-01 02:15:00+02:00   1296.80  entsoe_py_import  NO2_DE_LU_flow

  business_type         in_domain        out_domain unit resolution  position
0           A66  10Y1001A1001A82H  10YNO-2--------T  MAW      PT15M         1
1           A66  10Y1001A1001A82H  10YNO-2--------T  MAW      PT15M         2
```

```
---------------------------------------------------------------------------
Py4JJavaError                             Traceback (most recent call last)
Cell In[21], line 57
     53 (
```

```
     54     spark_df.write.format('org.apache.spark.sql.cassandra')
     55     .mode('append')
     56     .options(keyspace='norway_energy_data', table='cross_border_data')
---> 57     .save()
     58 )
     59
     60 print('Data successfully saved to Cassandra!')

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\sql\readwriter.py:1461, in DataFrameWriter.
 ↪save(self, path, format, mode, partitionBy, **options)
   1459     self.format(format)
   1460 if path is None:
-> 1461     self._jwrite.save()
   1462 else:
   1463     self._jwrite.save(path)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\java_gateway.py:1322, in JavaMember.__call__(self ␣
 ↪*args)
   1316 command = proto.CALL_COMMAND_NAME +\
   1317     self.command_header +\
   1318     args_command +\
   1319     proto.END_COMMAND_PART
   1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
   1323     answer, self.gateway_client, self.target_id, self.name)
   1325 for temp_arg in temp_args:
   1326     if hasattr(temp_arg, "_detach"):

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\errors\exceptions\captured.py:179, in␣
 ↪capture_sql_exception.<locals>.deco(*a, **kw)
    177 def deco(*a: Any, **kw: Any) -> Any:
    178     try:
--> 179         return f(*a, **kw)
    180     except Py4JJavaError as e:
    181         converted = convert_exception(e.java_exception)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\protocol.py:326, in get_return_value(answer,␣
 ↪gateway_client, target_id, name)
    324 value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    325 if answer[1] == REFERENCE_TYPE:
--> 326     raise Py4JJavaError(
    327         "An error occurred while calling {0}{1}{2}.\n".
    328         format(target_id, ".", name), value)
    329 else:
    330     raise Py4JError(
```

```
    331             "An error occurred while calling {0}{1}{2}. Trace:\n{3}\n".
    332             format(target_id, ".", name, value))

Py4JJavaError: An error occurred while calling o351.save.
: org.apache.spark.SparkException: Job aborted due to stage failure: Task 3 in␣
 ↪stage 0.0 failed 1 times, most recent failure: Lost task 3.0 in stage 0.0 (TI␣
 ↪3) (LAPTOP-ALLDEQIM.mshome.net executor driver): org.apache.spark.
 ↪SparkException: Python worker exited unexpectedly (crashed)

        at org.apache.spark.api.python.
 ↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:612

        at org.apache.spark.api.python.
 ↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:594

        at scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.
 ↪scala:38)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
 ↪scala:789)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
 ↪scala:766)

        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.
 ↪hasNext(PythonRunner.scala:525)

        at org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator
 ↪scala:37)

        at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at org.apache.spark.sql.catalyst.expressions.
 ↪GeneratedClass$GeneratedIteratorForCodegenStage1.processNext(Unknown Source)

        at org.apache.spark.sql.execution.BufferedRowIterator.
 ↪hasNext(BufferedRowIterator.java:43)

        at org.apache.spark.sql.execution.
 ↪WholeStageCodegenEvaluatorFactory$WholeStageCodegenPartitionEvaluator$$anon$1
 ↪hasNext(WholeStageCodegenEvaluatorFactory.scala:43)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
 ↪$anonfun$run$1(WriteToDataSourceV2Exec.scala:441)
```

```
        at org.apache.spark.util.Utils$.
↪tryWithSafeFinallyAndFailureCallbacks(Utils.scala:1397)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run(WriteToDataSourceV2Exec.scala:486)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run$(WriteToDataSourceV2Exec.scala:425)

        at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.
↪run(WriteToDataSourceV2Exec.scala:491)

        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪$anonfun$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)

        at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)

        at org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:
↪166)

        at org.apache.spark.scheduler.Task.run(Task.scala:141)

        at org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executo .
↪scala:620)

        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally(SparkErrorUtils.scala:64)

        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally$(SparkErrorUtils.scala:61)

        at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)

        at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623

        at java.base/java.util.concurrent.ThreadPoolExecutor.
↪runWorker(ThreadPoolExecutor.java:1136)

        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.
↪run(ThreadPoolExecutor.java:635)

        at java.base/java.lang.Thread.run(Thread.java:840)

Caused by: java.io.EOFException

        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)
```

```
        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:774)

        … 26 more


Driver stacktrace:

        at org.apache.spark.scheduler.DAGScheduler.
↪failJobAndIndependentStages(DAGScheduler.scala:2856)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$abortStage$2(DAGScheduler.scala:2792)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$abortStage$2$adapted(DAGScheduler.scala:2791)

        at scala.collection.mutable.ResizableArray.foreach(ResizableArray.scala
↪62)

        at scala.collection.mutable.ResizableArray.foreach$(ResizableArray.scal :
↪55)

        at scala.collection.mutable.ArrayBuffer.foreach(ArrayBuffer.scala:49)

        at org.apache.spark.scheduler.DAGScheduler.abortStage(DAGScheduler.scal :
↪2791)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$handleTaskSetFailed$1(DAGScheduler.scala:1247)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$handleTaskSetFailed$1$adapted(DAGScheduler.scala:1247)

        at scala.Option.foreach(Option.scala:407)

        at org.apache.spark.scheduler.DAGScheduler.
↪handleTaskSetFailed(DAGScheduler.scala:1247)

        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪doOnReceive(DAGScheduler.scala:3060)

        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪onReceive(DAGScheduler.scala:2994)

        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪onReceive(DAGScheduler.scala:2983)
```

```
        at org.apache.spark.util.EventLoop$$anon$1.run(EventLoop.scala:49)

        at org.apache.spark.scheduler.DAGScheduler.runJob(DAGScheduler.scala:98
        at org.apache.spark.SparkContext.runJob(SparkContext.scala:2398)

        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪writeWithV2(WriteToDataSourceV2Exec.scala:385)

        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪writeWithV2$(WriteToDataSourceV2Exec.scala:359)

        at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.
↪writeWithV2(WriteToDataSourceV2Exec.scala:225)

        at org.apache.spark.sql.execution.datasources.v2.
↪V2ExistingTableWriteExec.run(WriteToDataSourceV2Exec.scala:337)

        at org.apache.spark.sql.execution.datasources.v2.
↪V2ExistingTableWriteExec.run$(WriteToDataSourceV2Exec.scala:336)

        at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.
↪run(WriteToDataSourceV2Exec.scala:225)

        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪result$lzycompute(V2CommandExec.scala:43)

        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪result(V2CommandExec.scala:43)

        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪executeCollect(V2CommandExec.scala:49)

        at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.
↪$anonfun$applyOrElse$1(QueryExecution.scala:107)

        at org.apache.spark.sql.execution.SQLExecution$.
↪$anonfun$withNewExecutionId$6(SQLExecution.scala:125)

        at org.apache.spark.sql.execution.SQLExecution$.
↪withSQLConfPropagated(SQLExecution.scala:201)

        at org.apache.spark.sql.execution.SQLExecution$.
↪$anonfun$withNewExecutionId$1(SQLExecution.scala:108)

        at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:900)
```

```
	at org.apache.spark.sql.execution.SQLExecution$.
↪withNewExecutionId(SQLExecution.scala:66)

	at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(QueryExecution.
↪scala:107)

	at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(QueryExecution.
↪scala:98)

	at org.apache.spark.sql.catalyst.trees.TreeNode.
↪$anonfun$transformDownWithPruning$1(TreeNode.scala:461)

	at org.apache.spark.sql.catalyst.trees.CurrentOrigin$.withOrigin(origin
↪scala:76)

	at org.apache.spark.sql.catalyst.trees.TreeNode.
↪transformDownWithPruning(TreeNode.scala:461)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪org$apache$spark$sql$catalyst$plans$logical$AnalysisHelper$$super$transformDo nWithPruning
↪scala:32)

	at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.
↪transformDownWithPruning(AnalysisHelper.scala:267)

	at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.
↪transformDownWithPruning$(AnalysisHelper.scala:263)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪transformDownWithPruning(LogicalPlan.scala:32)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪transformDownWithPruning(LogicalPlan.scala:32)

	at org.apache.spark.sql.catalyst.trees.TreeNode.transformDown(TreeNode.
↪scala:437)

	at org.apache.spark.sql.execution.QueryExecution.
↪eagerlyExecuteCommands(QueryExecution.scala:98)

	at org.apache.spark.sql.execution.QueryExecution.
↪commandExecuted$lzycompute(QueryExecution.scala:85)

	at org.apache.spark.sql.execution.QueryExecution.
↪commandExecuted(QueryExecution.scala:83)
```

```
        at org.apache.spark.sql.execution.QueryExecution.
↪assertCommandExecuted(QueryExecution.scala:142)

        at org.apache.spark.sql.DataFrameWriter.runCommand(DataFrameWriter.scal:
↪859)

        at org.apache.spark.sql.DataFrameWriter.saveInternal(DataFrameWriter.
↪scala:312)

        at org.apache.spark.sql.DataFrameWriter.save(DataFrameWriter.scala:248)

        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
↪invoke0(Native Method)

        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
↪invoke(NativeMethodAccessorImpl.java:77)

        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.
↪invoke(DelegatingMethodAccessorImpl.java:43)

        at java.base/java.lang.reflect.Method.invoke(Method.java:569)

        at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)

        at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)

        at py4j.Gateway.invoke(Gateway.java:282)

        at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)

        at py4j.commands.CallCommand.execute(CallCommand.java:79)

        at py4j.ClientServerConnection.waitForCommands(ClientServerConnection.
↪java:182)

        at py4j.ClientServerConnection.run(ClientServerConnection.java:106)

        at java.base/java.lang.Thread.run(Thread.java:840)

Caused by: org.apache.spark.SparkException: Python worker exited unexpectedly␣
↪(crashed)

        at org.apache.spark.api.python.
↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:612

        at org.apache.spark.api.python.
↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:594
```

```
	at scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.
↪scala:38)

	at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:789)

	at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:766)

	at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.
↪hasNext(PythonRunner.scala:525)

	at org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator
↪scala:37)

	at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)

	at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

	at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

	at org.apache.spark.sql.catalyst.expressions.
↪GeneratedClass$GeneratedIteratorForCodegenStage1.processNext(Unknown Source)

	at org.apache.spark.sql.execution.BufferedRowIterator.
↪hasNext(BufferedRowIterator.java:43)

	at org.apache.spark.sql.execution.
↪WholeStageCodegenEvaluatorFactory$WholeStageCodegenPartitionEvaluator$$anon$1
↪hasNext(WholeStageCodegenEvaluatorFactory.scala:43)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪$anonfun$run$1(WriteToDataSourceV2Exec.scala:441)

	at org.apache.spark.util.Utils$.
↪tryWithSafeFinallyAndFailureCallbacks(Utils.scala:1397)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run(WriteToDataSourceV2Exec.scala:486)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run$(WriteToDataSourceV2Exec.scala:425)

	at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.
↪run(WriteToDataSourceV2Exec.scala:491)

	at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪$anonfun$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)
```

```
        at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)

        at org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:
↪166)

        at org.apache.spark.scheduler.Task.run(Task.scala:141)

        at org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executo .
↪scala:620)

        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally(SparkErrorUtils.scala:64)

        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally$(SparkErrorUtils.scala:61)

        at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)

        at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623

        at java.base/java.util.concurrent.ThreadPoolExecutor.
↪runWorker(ThreadPoolExecutor.java:1136)

        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.
↪run(ThreadPoolExecutor.java:635)

        … 1 more

Caused by: java.io.EOFException

        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:774)

        … 26 more
```

[20]:
```python
# Take only the first 5 rows for testing
df_mini = df_to_save.head(5).copy()

# Convert to Spark DataFrame
spark_mini_df = spark.createDataFrame(df_mini)
```

```
print("Attempting to write 5 test rows to Cassandra...")
try:
  (
      spark_mini_df.write.format("org.apache.spark.sql.cassandra")
      .mode("append")
      .options(keyspace="norway_energy_data", table="cross_border_data")
      .save()
  )
  print("Success! 5 rows written without crashing.")
except Exception as e:
  print("Detailed error:", e)
```

```
Attempting to write 5 test rows to Cassandra…
Detailed error: An error occurred while calling o287.save.
: org.apache.spark.SparkException: Job aborted due to stage failure: Task 10 in
stage 4.0 failed 1 times, most recent failure: Lost task 10.0 in stage 4.0 (TID
58) (LAPTOP-ALLDEQIM.mshome.net executor driver):
org.apache.spark.SparkException: Python worker exited unexpectedly (crashed)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator$$anonfun$
1.applyOrElse(PythonRunner.scala:612)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator$$anonfun$
1.applyOrElse(PythonRunner.scala:594)
        at
scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.scala:38)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:789)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:766)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.hasNext(P
ythonRunner.scala:525)
        at
org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator.scala:37)
        at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)
        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)
        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)
        at org.apache.spark.sql.catalyst.expressions.GeneratedClass$GeneratedIte
ratorForCodegenStage1.processNext(Unknown Source)
        at org.apache.spark.sql.execution.BufferedRowIterator.hasNext(BufferedRo
wIterator.java:43)
        at org.apache.spark.sql.execution.WholeStageCodegenEvaluatorFactory$Whol
eStageCodegenPartitionEvaluator$$anon$1.hasNext(WholeStageCodegenEvaluatorFactor
y.scala:43)
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.$anonf
un$run$1(WriteToDataSourceV2Exec.scala:441)
        at org.apache.spark.util.Utils$.tryWithSafeFinallyAndFailureCallbacks(Ut
ils.scala:1397)
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.run(Wr
iteToDataSourceV2Exec.scala:486)
```

```
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.run$(W
riteToDataSourceV2Exec.scala:425)
        at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.r
un(WriteToDataSourceV2Exec.scala:491)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.$anonf
un$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)
        at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)
        at
org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:166)
        at org.apache.spark.scheduler.Task.run(Task.scala:141)
        at
org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executor.scala:620)
        at org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally(SparkErrorUt
ils.scala:64)
        at org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally$(SparkErrorU
tils.scala:61)
        at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)
        at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoo
lExecutor.java:1136)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPo
olExecutor.java:635)
        at java.base/java.lang.Thread.run(Thread.java:840)
Caused by: java.io.EOFException
        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:774)
        … 26 more

Driver stacktrace:
        at org.apache.spark.scheduler.DAGScheduler.failJobAndIndependentStages(D
AGScheduler.scala:2856)
        at org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2(DAGSche
duler.scala:2792)
        at org.apache.spark.scheduler.DAGScheduler.$anonfun$abortStage$2$adapted
(DAGScheduler.scala:2791)
        at
scala.collection.mutable.ResizableArray.foreach(ResizableArray.scala:62)
        at
scala.collection.mutable.ResizableArray.foreach$(ResizableArray.scala:55)
        at scala.collection.mutable.ArrayBuffer.foreach(ArrayBuffer.scala:49)
        at
org.apache.spark.scheduler.DAGScheduler.abortStage(DAGScheduler.scala:2791)
        at org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$
1(DAGScheduler.scala:1247)
        at org.apache.spark.scheduler.DAGScheduler.$anonfun$handleTaskSetFailed$
1$adapted(DAGScheduler.scala:1247)
        at scala.Option.foreach(Option.scala:407)
```

```
        at org.apache.spark.scheduler.DAGScheduler.handleTaskSetFailed(DAGSchedu
ler.scala:1247)
        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.doOnReceive(D
AGScheduler.scala:3060)
        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAG
Scheduler.scala:2994)
        at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.onReceive(DAG
Scheduler.scala:2983)
        at org.apache.spark.util.EventLoop$$anon$1.run(EventLoop.scala:49)
        at
org.apache.spark.scheduler.DAGScheduler.runJob(DAGScheduler.scala:989)
        at org.apache.spark.SparkContext.runJob(SparkContext.scala:2398)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.writeW
ithV2(WriteToDataSourceV2Exec.scala:385)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.writeW
ithV2$(WriteToDataSourceV2Exec.scala:359)
        at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.writeWit
hV2(WriteToDataSourceV2Exec.scala:225)
        at org.apache.spark.sql.execution.datasources.v2.V2ExistingTableWriteExe
c.run(WriteToDataSourceV2Exec.scala:337)
        at org.apache.spark.sql.execution.datasources.v2.V2ExistingTableWriteExe
c.run$(WriteToDataSourceV2Exec.scala:336)
        at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.run(Writ
eToDataSourceV2Exec.scala:225)
        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.result$lz
ycompute(V2CommandExec.scala:43)
        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.result(V2
CommandExec.scala:43)
        at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.executeCo
llect(V2CommandExec.scala:49)
        at org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecute
Commands$1.$anonfun$applyOrElse$1(QueryExecution.scala:107)
        at org.apache.spark.sql.execution.SQLExecution$.$anonfun$withNewExecutio
nId$6(SQLExecution.scala:125)
        at org.apache.spark.sql.execution.SQLExecution$.withSQLConfPropagated(SQ
LExecution.scala:201)
        at org.apache.spark.sql.execution.SQLExecution$.$anonfun$withNewExecutio
nId$1(SQLExecution.scala:108)
        at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:900)
        at org.apache.spark.sql.execution.SQLExecution$.withNewExecutionId(SQLEx
ecution.scala:66)
        at org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecute
Commands$1.applyOrElse(QueryExecution.scala:107)
        at org.apache.spark.sql.execution.QueryExecution$$anonfun$eagerlyExecute
Commands$1.applyOrElse(QueryExecution.scala:98)
        at org.apache.spark.sql.catalyst.trees.TreeNode.$anonfun$transformDownWi
thPruning$1(TreeNode.scala:461)
        at
```

```
org.apache.spark.sql.catalyst.trees.CurrentOrigin$.withOrigin(origin.scala:76)
        at org.apache.spark.sql.catalyst.trees.TreeNode.transformDownWithPruning
(TreeNode.scala:461)
        at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.org$apache$sp
ark$sql$catalyst$plans$logical$AnalysisHelper$$super$transformDownWithPruning(Lo
gicalPlan.scala:32)
        at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.transformD
ownWithPruning(AnalysisHelper.scala:267)
        at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.transformD
ownWithPruning$(AnalysisHelper.scala:263)
        at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.transformDown
WithPruning(LogicalPlan.scala:32)
        at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.transformDown
WithPruning(LogicalPlan.scala:32)
        at
org.apache.spark.sql.catalyst.trees.TreeNode.transformDown(TreeNode.scala:437)
        at org.apache.spark.sql.execution.QueryExecution.eagerlyExecuteCommands(
QueryExecution.scala:98)
        at org.apache.spark.sql.execution.QueryExecution.commandExecuted$lzycomp
ute(QueryExecution.scala:85)
        at org.apache.spark.sql.execution.QueryExecution.commandExecuted(QueryEx
ecution.scala:83)
        at org.apache.spark.sql.execution.QueryExecution.assertCommandExecuted(Q
ueryExecution.scala:142)
        at
org.apache.spark.sql.DataFrameWriter.runCommand(DataFrameWriter.scala:859)
        at
org.apache.spark.sql.DataFrameWriter.saveInternal(DataFrameWriter.scala:312)
        at org.apache.spark.sql.DataFrameWriter.save(DataFrameWriter.scala:248)
        at
java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke0(Native Method)
        at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.invoke(Native
MethodAccessorImpl.java:77)
        at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.invoke(De
legatingMethodAccessorImpl.java:43)
        at java.base/java.lang.reflect.Method.invoke(Method.java:569)
        at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)
        at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)
        at py4j.Gateway.invoke(Gateway.java:282)
        at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)
        at py4j.commands.CallCommand.execute(CallCommand.java:79)
        at
py4j.ClientServerConnection.waitForCommands(ClientServerConnection.java:182)
        at py4j.ClientServerConnection.run(ClientServerConnection.java:106)
        at java.base/java.lang.Thread.run(Thread.java:840)
Caused by: org.apache.spark.SparkException: Python worker exited unexpectedly
(crashed)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator$$anonfun$
```

```
1.applyOrElse(PythonRunner.scala:612)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator$$anonfun$
1.applyOrElse(PythonRunner.scala:594)
        at
scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.scala:38)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:789)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:766)
        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.hasNext(P
ythonRunner.scala:525)
        at
org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator.scala:37)
        at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)
        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)
        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)
        at org.apache.spark.sql.catalyst.expressions.GeneratedClass$GeneratedIte
ratorForCodegenStage1.processNext(Unknown Source)
        at org.apache.spark.sql.execution.BufferedRowIterator.hasNext(BufferedRo
wIterator.java:43)
        at org.apache.spark.sql.execution.WholeStageCodegenEvaluatorFactory$Whol
eStageCodegenPartitionEvaluator$$anon$1.hasNext(WholeStageCodegenEvaluatorFactor
y.scala:43)
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.$anonf
un$run$1(WriteToDataSourceV2Exec.scala:441)
        at org.apache.spark.util.Utils$.tryWithSafeFinallyAndFailureCallbacks(Ut
ils.scala:1397)
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.run(Wr
iteToDataSourceV2Exec.scala:486)
        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.run$(W
riteToDataSourceV2Exec.scala:425)
        at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.r
un(WriteToDataSourceV2Exec.scala:491)
        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.$anonf
un$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)
        at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)
        at
org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:166)
        at org.apache.spark.scheduler.Task.run(Task.scala:141)
        at
org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executor.scala:620)
        at org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally(SparkErrorUt
ils.scala:64)
        at org.apache.spark.util.SparkErrorUtils.tryWithSafeFinally$(SparkErrorU
tils.scala:61)
        at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)
        at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623)
        at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoo
```

```
lExecutor.java:1136)
        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPo
olExecutor.java:635)
        … 1 more
Caused by: java.io.EOFException
        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)
        at
org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.scala:774)
        … 26 more
```

[24]:
```python
from pyspark.sql import SparkSession
import os
import sys

# CRITICAL: Force Spark to use the exact python executable from your active␣
 ↪virtual environment (.venv)
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable
os.environ["HADOOP_HOME"] = "C:/Hadoop/hadoop-3.3.1"

# Stop any existing session
try:
  spark.stop()
except:
  pass

# Initialize Spark session with Arrow disabled and correct paths
spark = (
    SparkSession.builder.appName("CassandraConnectionTest")
    .config(
        "spark.jars.packages",
        "com.datastax.spark:spark-cassandra-connector_2.12:3.5.1",
    )
    .config("spark.cassandra.connection.host", "127.0.0.1")
    .config("spark.cassandra.connection.port", "9042")
    .config("spark.sql.execution.arrow.pyspark.enabled", "false")
    .getOrCreate()
)

print("Spark session initialized using Python:", sys.executable)

# Test writing your mini dataframe again
df_mini = df_to_save.head(5).copy()
spark_mini_df = spark.createDataFrame(df_mini)

print("Attempting to write 5 test rows to Cassandra...")
```

```
(
    spark_mini_df.write.format("org.apache.spark.sql.cassandra")
    .mode("append")
    .options(keyspace="norway_energy_data", table="cross_border_data")
    .save()
)

print("Success! 5 test rows written to Cassandra!")
```

```
Spark session initialized using Python: c:\Users\emryc\Documents\Dokumenter\NMBU
\Kurs\_IND320\IND320_project\.venv\Scripts\python.exe
Attempting to write 5 test rows to Cassandra…
```

```
---------------------------------------------------------------------------
Py4JJavaError                             Traceback (most recent call last)
Cell In[24], line 40
     36 (
     37     spark_mini_df.write.format("org.apache.spark.sql.cassandra")
     38     .mode("append")
     39     .options(keyspace="norway_energy_data", table="cross_border_data")
---> 40     .save()
     41 )
     42
     43 print("Success! 5 test rows written to Cassandra!")

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\sql\readwriter.py:1461, in DataFrameWriter.
 ↪save(self, path, format, mode, partitionBy, **options)
   1459     self.format(format)
   1460 if path is None:
-> 1461     self._jwrite.save()
   1462 else:
   1463     self._jwrite.save(path)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\java_gateway.py:1322, in JavaMember.__call__(self
 ↪*args)
   1316 command = proto.CALL_COMMAND_NAME +\
   1317     self.command_header +\
   1318     args_command +\
   1319     proto.END_COMMAND_PART
   1321 answer = self.gateway_client.send_command(command)
-> 1322 return_value = get_return_value(
   1323     answer, self.gateway_client, self.target_id, self.name)
   1325 for temp_arg in temp_args:
   1326     if hasattr(temp_arg, "_detach"):
```

```
File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\pyspark\errors\exceptions\captured.py:179, in␣
 ↪capture_sql_exception.<locals>.deco(*a, **kw)
    177 def deco(*a: Any, **kw: Any) -> Any:
    178     try:
--> 179         return f(*a, **kw)
    180     except Py4JJavaError as e:
    181         converted = convert_exception(e.java_exception)

File ~\Documents\Dokumenter\NMBU\Kurs\_IND320\IND320_project\.
 ↪venv\Lib\site-packages\py4j\protocol.py:326, in get_return_value(answer,␣
 ↪gateway_client, target_id, name)
    324 value = OUTPUT_CONVERTER[type](answer[2:], gateway_client)
    325 if answer[1] == REFERENCE_TYPE:
--> 326     raise Py4JJavaError(
    327         "An error occurred while calling {0}{1}{2}.\n".
    328         format(target_id, ".", name), value)
    329 else:
    330     raise Py4JError(
    331         "An error occurred while calling {0}{1}{2}. Trace:\n{3}\n".
    332         format(target_id, ".", name, value))

Py4JJavaError: An error occurred while calling o483.save.
: org.apache.spark.SparkException: Job aborted due to stage failure: Task 11 in␣
 ↪stage 0.0 failed 1 times, most recent failure: Lost task 11.0 in stage 0.0␣
 ↪(TID 11) (LAPTOP-ALLDEQIM.mshome.net executor driver): org.apache.spark.
 ↪SparkException: Python worker exited unexpectedly (crashed)

        at org.apache.spark.api.python.
 ↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:612

        at org.apache.spark.api.python.
 ↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:594

        at scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.
 ↪scala:38)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
 ↪scala:789)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
 ↪scala:766)

        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.
 ↪hasNext(PythonRunner.scala:525)

        at org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator
 ↪scala:37)
```

```
        at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at org.apache.spark.sql.catalyst.expressions.
↪GeneratedClass$GeneratedIteratorForCodegenStage1.processNext(Unknown Source)

        at org.apache.spark.sql.execution.BufferedRowIterator.
↪hasNext(BufferedRowIterator.java:43)

        at org.apache.spark.sql.execution.
↪WholeStageCodegenEvaluatorFactory$WholeStageCodegenPartitionEvaluator$$anon$1
↪hasNext(WholeStageCodegenEvaluatorFactory.scala:43)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪$anonfun$run$1(WriteToDataSourceV2Exec.scala:441)

        at org.apache.spark.util.Utils$.
↪tryWithSafeFinallyAndFailureCallbacks(Utils.scala:1397)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run(WriteToDataSourceV2Exec.scala:486)

        at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run$(WriteToDataSourceV2Exec.scala:425)

        at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.
↪run(WriteToDataSourceV2Exec.scala:491)

        at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪$anonfun$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)

        at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)

        at org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:
↪166)

        at org.apache.spark.scheduler.Task.run(Task.scala:141)

        at org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executo
↪scala:620)

        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally(SparkErrorUtils.scala:64)
```

```
        at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally$(SparkErrorUtils.scala:61)

        at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)

        at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623

        at java.base/java.util.concurrent.ThreadPoolExecutor.
↪runWorker(ThreadPoolExecutor.java:1136)

        at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.
↪run(ThreadPoolExecutor.java:635)

        at java.base/java.lang.Thread.run(Thread.java:840)

Caused by: java.io.EOFException

        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:774)

        … 26 more


Driver stacktrace:

        at org.apache.spark.scheduler.DAGScheduler.
↪failJobAndIndependentStages(DAGScheduler.scala:2856)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$abortStage$2(DAGScheduler.scala:2792)

        at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$abortStage$2$adapted(DAGScheduler.scala:2791)

        at scala.collection.mutable.ResizableArray.foreach(ResizableArray.scala
↪62)

        at scala.collection.mutable.ResizableArray.foreach$(ResizableArray.scal
↪55)

        at scala.collection.mutable.ArrayBuffer.foreach(ArrayBuffer.scala:49)

        at org.apache.spark.scheduler.DAGScheduler.abortStage(DAGScheduler.scal
↪2791)
```

```
	at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$handleTaskSetFailed$1(DAGScheduler.scala:1247)

	at org.apache.spark.scheduler.DAGScheduler.
↪$anonfun$handleTaskSetFailed$1$adapted(DAGScheduler.scala:1247)

	at scala.Option.foreach(Option.scala:407)

	at org.apache.spark.scheduler.DAGScheduler.
↪handleTaskSetFailed(DAGScheduler.scala:1247)

	at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪doOnReceive(DAGScheduler.scala:3060)

	at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪onReceive(DAGScheduler.scala:2994)

	at org.apache.spark.scheduler.DAGSchedulerEventProcessLoop.
↪onReceive(DAGScheduler.scala:2983)

	at org.apache.spark.util.EventLoop$$anon$1.run(EventLoop.scala:49)

	at org.apache.spark.scheduler.DAGScheduler.runJob(DAGScheduler.scala:98 )

	at org.apache.spark.SparkContext.runJob(SparkContext.scala:2398)

	at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪writeWithV2(WriteToDataSourceV2Exec.scala:385)

	at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪writeWithV2$(WriteToDataSourceV2Exec.scala:359)

	at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.
↪writeWithV2(WriteToDataSourceV2Exec.scala:225)

	at org.apache.spark.sql.execution.datasources.v2.
↪V2ExistingTableWriteExec.run(WriteToDataSourceV2Exec.scala:337)

	at org.apache.spark.sql.execution.datasources.v2.
↪V2ExistingTableWriteExec.run$(WriteToDataSourceV2Exec.scala:336)

	at org.apache.spark.sql.execution.datasources.v2.AppendDataExec.
↪run(WriteToDataSourceV2Exec.scala:225)

	at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪result$lzycompute(V2CommandExec.scala:43)
```

```
	at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪result(V2CommandExec.scala:43)

	at org.apache.spark.sql.execution.datasources.v2.V2CommandExec.
↪executeCollect(V2CommandExec.scala:49)

	at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.
↪$anonfun$applyOrElse$1(QueryExecution.scala:107)

	at org.apache.spark.sql.execution.SQLExecution$.
↪$anonfun$withNewExecutionId$6(SQLExecution.scala:125)

	at org.apache.spark.sql.execution.SQLExecution$.
↪withSQLConfPropagated(SQLExecution.scala:201)

	at org.apache.spark.sql.execution.SQLExecution$.
↪$anonfun$withNewExecutionId$1(SQLExecution.scala:108)

	at org.apache.spark.sql.SparkSession.withActive(SparkSession.scala:900)

	at org.apache.spark.sql.execution.SQLExecution$.
↪withNewExecutionId(SQLExecution.scala:66)

	at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(QueryExecution.
↪scala:107)

	at org.apache.spark.sql.execution.
↪QueryExecution$$anonfun$eagerlyExecuteCommands$1.applyOrElse(QueryExecution.
↪scala:98)

	at org.apache.spark.sql.catalyst.trees.TreeNode.
↪$anonfun$transformDownWithPruning$1(TreeNode.scala:461)

	at org.apache.spark.sql.catalyst.trees.CurrentOrigin$.withOrigin(origin
↪scala:76)

	at org.apache.spark.sql.catalyst.trees.TreeNode.
↪transformDownWithPruning(TreeNode.scala:461)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪org$apache$spark$sql$catalyst$plans$logical$AnalysisHelper$$super$transformDownWithPruning
↪scala:32)

	at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.
↪transformDownWithPruning(AnalysisHelper.scala:267)
```

```
	at org.apache.spark.sql.catalyst.plans.logical.AnalysisHelper.
↪transformDownWithPruning$(AnalysisHelper.scala:263)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪transformDownWithPruning(LogicalPlan.scala:32)

	at org.apache.spark.sql.catalyst.plans.logical.LogicalPlan.
↪transformDownWithPruning(LogicalPlan.scala:32)

	at org.apache.spark.sql.catalyst.trees.TreeNode.transformDown(TreeNode.
↪scala:437)

	at org.apache.spark.sql.execution.QueryExecution.
↪eagerlyExecuteCommands(QueryExecution.scala:98)

	at org.apache.spark.sql.execution.QueryExecution.
↪commandExecuted$lzycompute(QueryExecution.scala:85)

	at org.apache.spark.sql.execution.QueryExecution.
↪commandExecuted(QueryExecution.scala:83)

	at org.apache.spark.sql.execution.QueryExecution.
↪assertCommandExecuted(QueryExecution.scala:142)

	at org.apache.spark.sql.DataFrameWriter.runCommand(DataFrameWriter.scal:
↪859)

	at org.apache.spark.sql.DataFrameWriter.saveInternal(DataFrameWriter.
↪scala:312)

	at org.apache.spark.sql.DataFrameWriter.save(DataFrameWriter.scala:248)

	at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
↪invoke0(Native Method)

	at java.base/jdk.internal.reflect.NativeMethodAccessorImpl.
↪invoke(NativeMethodAccessorImpl.java:77)

	at java.base/jdk.internal.reflect.DelegatingMethodAccessorImpl.
↪invoke(DelegatingMethodAccessorImpl.java:43)

	at java.base/java.lang.reflect.Method.invoke(Method.java:569)

	at py4j.reflection.MethodInvoker.invoke(MethodInvoker.java:244)

	at py4j.reflection.ReflectionEngine.invoke(ReflectionEngine.java:374)

	at py4j.Gateway.invoke(Gateway.java:282)
```

```
        at py4j.commands.AbstractCommand.invokeMethod(AbstractCommand.java:132)

        at py4j.commands.CallCommand.execute(CallCommand.java:79)

        at py4j.ClientServerConnection.waitForCommands(ClientServerConnection.
↪java:182)

        at py4j.ClientServerConnection.run(ClientServerConnection.java:106)

        at java.base/java.lang.Thread.run(Thread.java:840)

Caused by: org.apache.spark.SparkException: Python worker exited unexpectedly␣
↪(crashed)

        at org.apache.spark.api.python.
↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:612

        at org.apache.spark.api.python.
↪BasePythonRunner$ReaderIterator$$anonfun$1.applyOrElse(PythonRunner.scala:594

        at scala.runtime.AbstractPartialFunction.apply(AbstractPartialFunction.
↪scala:38)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:789)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
↪scala:766)

        at org.apache.spark.api.python.BasePythonRunner$ReaderIterator.
↪hasNext(PythonRunner.scala:525)

        at org.apache.spark.InterruptibleIterator.hasNext(InterruptibleIterator
↪scala:37)

        at scala.collection.Iterator$$anon$11.hasNext(Iterator.scala:491)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)

        at org.apache.spark.sql.catalyst.expressions.
↪GeneratedClass$GeneratedIteratorForCodegenStage1.processNext(Unknown Source)

        at org.apache.spark.sql.execution.BufferedRowIterator.
↪hasNext(BufferedRowIterator.java:43)
```

```
	at org.apache.spark.sql.execution.
↪WholeStageCodegenEvaluatorFactory$WholeStageCodegenPartitionEvaluator$$anon$1
↪hasNext(WholeStageCodegenEvaluatorFactory.scala:43)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪$anonfun$run$1(WriteToDataSourceV2Exec.scala:441)

	at org.apache.spark.util.Utils$.
↪tryWithSafeFinallyAndFailureCallbacks(Utils.scala:1397)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run(WriteToDataSourceV2Exec.scala:486)

	at org.apache.spark.sql.execution.datasources.v2.WritingSparkTask.
↪run$(WriteToDataSourceV2Exec.scala:425)

	at org.apache.spark.sql.execution.datasources.v2.DataWritingSparkTask$.
↪run(WriteToDataSourceV2Exec.scala:491)

	at org.apache.spark.sql.execution.datasources.v2.V2TableWriteExec.
↪$anonfun$writeWithV2$2(WriteToDataSourceV2Exec.scala:388)

	at org.apache.spark.scheduler.ResultTask.runTask(ResultTask.scala:93)

	at org.apache.spark.TaskContext.runTaskWithListeners(TaskContext.scala:
↪166)

	at org.apache.spark.scheduler.Task.run(Task.scala:141)

	at org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$4(Executo .
↪scala:620)

	at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally(SparkErrorUtils.scala:64)

	at org.apache.spark.util.SparkErrorUtils.
↪tryWithSafeFinally$(SparkErrorUtils.scala:61)

	at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:94)

	at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:623

	at java.base/java.util.concurrent.ThreadPoolExecutor.
↪runWorker(ThreadPoolExecutor.java:1136)

	at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.
↪run(ThreadPoolExecutor.java:635)
```

```
        … 1 more

Caused by: java.io.EOFException

        at java.base/java.io.DataInputStream.readInt(DataInputStream.java:386)

        at org.apache.spark.api.python.PythonRunner$$anon$3.read(PythonRunner.
 ↪scala:774)

        … 26 more
```

[ ]:
```
# Testing if cassandra works. Using teachers' code.
# This cell needs the cassandra container in the docker desktop running in the␣
 ↪backgroun to work.
cluster = Cluster(['localhost'], port=9042)
session = cluster.connect()

# I learned this one from gemini actually. 'try/finally' seems useful for this.
try:
    # Querying the cassandra version to check the connection.
    row = session.execute("SELECT release_version FROM system.local;").one()
    print(f"Successfully connected! Cassandra version: {row.release_version}")
finally:
    # I wanted to shutdown the connection here just in case.
    session.shutdown()
    cluster.shutdown()
    print("Connection closed cleanly.")
```

```
Successfully connected! Cassandra version: 5.0.9
Connection closed cleanly.
```

[23]:
```
# Close the connection when done
cluster.shutdown()
```

[ ]:
```
# Strip XML namespaces for easier querying
def strip_namespace(element):
  for elem in element.iter():
    if '}' in elem.tag:
      elem.tag = elem.tag.split('}', 1)[1]
  return elem
```

[ ]:
```
root = ET.fromstring(response.content)
root = strip_namespace(root)

doc_mrid = root.findtext('mRID')
```

```
records = []
for ts in root.findall('TimeSeries'):
  ts_mrid = ts.findtext('mRID')
  business_type = ts.findtext('businessType')
  in_domain = ts.findtext('in_Domain.mRID')
  out_domain = ts.findtext('out_Domain.mRID')
  unit = ts.findtext('quantity_Measure_Unit.name')

  for period in ts.findall('Period'):
    resolution = period.findtext('resolution')
    period_start = period.findtext('timeInterval/start')

    for point in period.findall('Point'):
      position = int(point.findtext('position'))
      quantity = float(point.findtext('quantity'))

      records.append({
          'document_mrid': doc_mrid,
          'timeseries_mrid': ts_mrid,
          'business_type': business_type,
          'in_domain': in_domain,
          'out_domain': out_domain,
          'unit': unit,
          'resolution': resolution,
          'period_start': period_start,
          'position': position,
          'quantity': quantity,
      })

print(f'Successfully parsed {len(records)} time-series data points.')
```

```
Successfully parsed 0 time-series data points.
```

[ ]:
```
root = ET.fromstring(response.content)


def strip_namespace(element):
  for elem in element.iter():
    if '}' in elem.tag:
      elem.tag = elem.tag.split('}', 1)[1]
  return elem


root = strip_namespace(root)

timeseries_list = root.findall('TimeSeries')
```

```python
print(f'1. Found {len(timeseries_list)} TimeSeries elements.')

for i, ts in enumerate(timeseries_list):
  periods = ts.findall('Period')
  print(f'   - TimeSeries {i}: Found {len(periods)} Period elements.')

  for j, period in enumerate(periods):
    points = period.findall('Point')
    print(f'     - Period {j}: Found {len(points)} Point elements.')
```

```
1. Found 0 TimeSeries elements.
```